MODULE 2.2

Application Security

Network Defense | Cisco Networking Academy

Prepared by: Kudzaishe Majeza

Cisco Networking Academy | 2026

Topics Covered:

2.2.1 – Application Security Overview

2.2.2 – Application Development Process

2.2.3 – Secure Coding Techniques

2.2.5 / 2.2.6 – Input Validation and Validation Rules

2.2.7 – Integrity Checks

2.2.8 / 2.2.10 – Other Practices & Managing Threats

2.2.1 Application Security Overview

Protecting the applications, websites and online services your organisation develops and uses

What is Application Security?

- Involves protecting applications, websites and online services from cyber threats
- Security must be a top priority during development, testing and deployment
- A single vulnerability in an application can expose an entire organisation's data
- Threats can come from both external attackers and insecure internal coding practices

Why It Matters

Data Breaches	Attackers exploit app flaws to steal sensitive user and company data
Financial Loss	Successful attacks lead to downtime, fines and recovery costs
Reputation Damage	Breaches erode customer trust and damage brand reputation
Compliance Risk	Unprotected apps violate regulations (GDPR, PCI-DSS, HIPAA)
Service Disruption	Vulnerable apps are targeted by DoS and ransomware attacks

2.2.2 Application Development

Maintaining security at all stages of the development lifecycle

The Secure Development Principle

- Security must be built in from the start — not added after the fact
- A robust development process ensures security requirements are met at every phase
- Testing should simulate real-world attacks before software is deployed
- All team members — developers, testers and ops — share responsibility for security

Secure Development Lifecycle Stages

1 Plan	Define security requirements and risk assessments upfront
2 Design	Architect the system with threat modelling and secure design patterns
3 Code	Apply secure coding techniques and code review processes
4 Test	Conduct penetration testing, vulnerability scanning and QA checks
5 Deploy	Use secure configurations, certificates and access controls
6 Monitor	Continuously monitor for new vulnerabilities and apply patches

2.2.3 Secure Coding Techniques

Developers use these techniques to validate that all security requirements have been met

Normalization & Stored Procedures

Normalization
Organises data to maintain integrity. Converts input strings to their simplest known form to ensure unique binary representations and identify malicious input.

Stored Procedures
Precompiled SQL statements stored in a database. Accepting input via stored procedures reduces network traffic and helps prevent SQL injection attacks.

Code Reuse
Using existing software to build new software saves time. Careful review is needed to avoid inheriting vulnerabilities from reused code.

Obfuscation, Camouflage & SDKs

Obfuscation & Camouflage
Obfuscation hides original data with random characters to prevent reverse engineering. Camouflage replaces sensitive data with realistic fictional data.

SDKs & Third-Party Libraries
Provide a repository of useful code to speed up development. However, any vulnerabilities in SDKs or third-party libraries can potentially affect many applications.

Key Principle
Always review third-party code for vulnerabilities, keep SDKs updated, and limit code reuse to well-vetted, actively maintained sources.

2.2.5 & 2.2.6 Input Validation and Validation Rules

Controlling data input is key to maintaining database integrity

Input Validation

- Many attacks insert malformed data to confuse, crash or expose applications
- Proper input validation stops bad data before it reaches the database

Attack Example: Newsletter Flooding

1. Customer fills out a web form to subscribe to a newsletter
2. Database generates and sends email with confirmation URL
3. Attacker modifies the URL to change username, email or subscription status
4. Host server receives bogus information without detecting it
5. Attacker automates this to flood the database with thousands of invalid subscribers

Validation Rules

A validation rule checks data falls within parameters defined by the database designer.

Size	Checks the number of characters in a data item
Format	Checks that data conforms to a specified format
Consistency	Checks codes in related data items are consistent
Range	Checks data lies within a minimum and maximum value
Check Digit	Extra calculation to generate a check digit for error detection

2.2.7 Integrity Checks

Ensuring data has not been altered or corrupted during storage or transmission

What are Integrity Checks?

- Measures the consistency of data in a file, picture or record
- Uses a hash function to take a snapshot of data at a point in time
- If data changes, the resulting hash will be different — revealing tampering

Common Hash Functions:

- MD5
- SHA-1
- SHA-256
- SHA-512

These compare data to a hashed value using complex mathematical algorithms.

Integrity Assurance Methods

Checksum	Converts data to a sum; receiver repeats process — if sums match, data is valid
Hash Functions	Download hash values from source are compared with locally generated hashes
Version Control	Prevents two users updating the same file simultaneously; others get read-only access
Backups	Accurate backups restore integrity when data becomes corrupted or lost
Authorization	File permissions and user access controls ensure only authorised users modify data

2.2.8 Other Application Security Practices

Additional techniques to ensure software authenticity and secure browsing

Code Signing

- Helps prove that a piece of software is authentic
- Executables are digitally signed to validate the author's identity
- Provides assurance that the software code has not changed since it was signed
- Operating systems can warn users or block software that is unsigned
- Certificate Authorities (CAs) issue code-signing certificates to verified publishers
- Prevents malware from impersonating legitimate software installers

Secure Cookies

- Protects information stored in browser cookies from hackers
- When a client interacts with a server, the server sends an HTTP response instructing the browser to create a cookie
- The cookie stores data for future requests during your browsing session
- Web developers must use cookies with HTTPS to prevent transmission over unencrypted HTTP
- The 'Secure' and 'HttpOnly' flags further protect cookies from interception and script access
- SameSite attribute prevents cross-site request forgery (CSRF) attacks

2.2.10 Managing Threats to Applications

Countermeasures organisations can implement to manage application domain threats

Access & Availability

Unauthorized Access to Data Centers

Implement policies, standards and procedures for staff and visitors to ensure facilities are secure.

Server and System Downtime

Develop a business continuity plan for critical applications.
Develop a disaster recovery plan for critical applications and data.

System Vulnerabilities

Network OS Software Vulnerability

Develop a policy to address application and OS updates.
Install patches and updates regularly.

Unauthorized Access to Systems

Use multi-factor authentication.
Monitor log files for suspicious activity.

Data & Development

Data Loss

Implement data classification standards.
Implement backup procedures and verify their integrity.

Software Development Vulnerabilities

Conduct thorough software testing prior to launch.
Use secure coding techniques throughout development.

2.2 Summary – Application Security

Key concepts from this module

Secure Coding

- Normalization
- Stored Procedures
- Obfuscation & Camouflage
- Code Reuse (carefully)
- SDK security awareness

Data Integrity

- Input Validation
- Validation Rules (Size, Format, Consistency, Range, Check Digit)
- Integrity Checks (Hash / Checksum)
- Version Control
- Authorisation & Backups

Application Protection

- Code Signing
- Secure Cookies (HTTPS, Secure flag)
- Policies for Data Center Access
- Business Continuity Planning
- Regular Patching & MFA